

Programación Genética: De la Mochila a Código

https://gemini.google.com/app/2244e8df3008bcde

User prompt: Bien. Ya como último trabajo. Necesito hacer un video. Con todo lo siguiente: Dame un guión con líneas de tiempo y dialogos. Además de mostrar imagenes y así.

Response: ¡Claro que sí, Ángel! Vamos a armar este guion para tu "Tesis en 3 Minutos" (3MT). Este formato exige ser sumamente directos, enganchar a la audiencia visualmente y hablar con mucha seguridad.

El guion está calculado para ~420 palabras, lo que a un ritmo de habla normal se traduce exactamente en un margen seguro de 2 minutos con 45 segundos a 3 minutos. La rúbrica exige que todos participen, así que dividí los diálogos asumiendo un equipo de 3 a 4 personas. Si son menos, simplemente repartan los bloques.

Aquí tienes tu guion estructurado al milímetro con apoyos visuales:

Guion de Video: Tesis en 3 Minutos (3MT)

Tema: Evolución Autónoma de Reglas de Asignación con Programación Genética Tipada

Tiempo	Secciones & Presentador	Diálogo (Texto a memorizar/leer)	Apoyo Visual en Pantalla (Imágenes/Video)
0:00 - 0:25 (25 seg)	1. Problema (Presentador 1)	"Imagina que un servidor en la nube recibe un pico masivo de tráfico. En la gestión de Tecnologías Computacionales, la asignación de recursos tiene que ser perfecta y en fracciones de segundo. El problema es que los algoritmos estáticos tradicionales fracasan aquí; son rígidos, preprogramados por humanos y colapsan cuando los datos operativos cambian de forma impredecible y caótica."	Visual: Animación rápida de un servidor colapsando o un gráfico de tráfico rebasando el límite. Texto en grande: EL PROBLEMA: Algoritmos Estáticos vs Entornos Caóticos.
0:25 - 0:50 (25 seg)	2. Solución / Hipótesis (Presentador 2)	"Nuestra hipótesis propone que el software puede evolucionar para escribir su propio código. Proponemos un sistema de Programación Genética Tipada que, en lugar de evadir el desorden, se alimenta de él. Si combinamos transferencia de aprendizaje continuo con una penalización estricta al crecimiento del código, el sistema generará de forma autónoma ecuaciones matemáticas limpias, adaptables y superiores al diseño manual."	Visual: Animación de un árbol sintáctico formándose rápidamente. Texto en grande: LA SOLUCIÓN: Programación Genética Tipada + Control Geométrico.
0:50 - 1:05 (15 seg)	3. Objetivos (Presentador 3)	"Nuestro objetivo principal fue desarrollar un motor evolutivo capaz de resolver el Problema de la Mochila multi-instancia bajo estrés extremo, garantizando reglas de decisión generalizables, minimizando el consumo de memoria y erradicando el código basura redundante."	Visual: <i>Bullet points</i> concisos con iconos. 1. Motor Evolutivo. 2. Generalización Multi-Instancia. 3. Cero Código Basura.
1:05 - 1:50 (45 seg)	4. Metodología (Presentador 1)	"Utilizamos el framework DEAP en Python para simular 20 fases de estrés continuo. Sometimos a la población a mochilas y objetos totalmente variables. La clave metodológica fue la restricción: limitamos la altura del árbol a 8 niveles e implementamos un castigo aritmético por longitud. Descartamos intencionalmente técnicas de perfilamiento previo, exponiendo al algoritmo al entorno crudo para forzar una generalización real."	Visual: Capturas breves del código en Python (DEAP), enfocando la línea del <code>gp.staticLimit</code> y el <code>evaluar_hiper_heuristica</code> . Un ícono tachando herramientas de <i>clustering</i> .
1:50 - 2:35 (45 seg)	5. Resultados (Presentador 2)	"Los resultados demostraron una resiliencia algorítmica rotunda. A través del análisis con WEKA, nuestras gráficas de dispersión multiclasex evidenciaron que, ante un choque ambiental, el motor recupera la estabilidad en menos de 3 generaciones. Además, el modelo de árbol J48 comprobó matemáticamente que las heurísticas evolucionadas logran saturar la ganancia máxima de manera sistemática, mientras que la penalización contrajo con éxito la franja de caos, creando expresiones matemáticamente óptimas."	Visual: ¡Momento de brillar! Muestra la gráfica verde de "Convergencia Evolutiva" (la de la Fase 5). Luego, cambia rápidamente a una toma parcial del Árbol J48 de WEKA resaltando un nodo clave.
2:35 - 3:00 (25 seg)	6. Conclusiones (Presentador 3)	"Concluimos que la evolución de software supera la creación estática. Logramos una inteligencia artificial verdaderamente autónoma que produce código transparente, rápido y sin consumo excesivo de memoria. Delegar la lógica operativa a un sistema evolutivo es un paradigma viable para optimizar microservicios y gestionar las infraestructuras de alto valor del mañana. Gracias."	Visual: Toma de todos los presentadores (o pantalla dividida). Texto final: CONCLUSIÓN: El futuro del software es evolutivo, no estático. Nombres de los integrantes del equipo.

Tips rápidos para la grabación:

- **Ensayen con cronómetro:** Lean el texto en voz alta. Si se pasan de los 3 minutos, hablen un 10% más rápido o corten alguna palabra de unión, pero el guion base ya está calibrado.
- **Energía:** Las partes 1 y 5 (Problema y Resultados) deben decirse con la mayor fuerza vocal y convicción.

- **Imágenes:** Utiliza las capturas exactas que procesamos juntos en WEKA y la gráfica de Python; son visualmente impresionantes y suben mucho el nivel técnico de una presentación en video. ¡Éxito con la edición!

User prompt: Pero no hables en plural, habla tipo: "El objetivo", "la hipótesis", "el software", "el desarrollo

Response: ¡Entendido, Ángel! Tienes toda la razón. Para mantener la formalidad y objetividad que requiere una presentación académica de tesis, es mucho mejor utilizar la tercera persona o la voz pasiva, eliminando por completo el "nosotros" o "nuestro".

He reescrito los diálogos del guion adoptando exactamente el tono impersonal y directo que me pides ("El objetivo", "la hipótesis", "el desarrollo").

Aquí tienes el guion ajustado:

Guion de Video: Tesis en 3 Minutos (3MT)

Tema: Optimización Dinámica y Evolución de Reglas de Asignación

Tiempo	Secciones & Presentador	Diálogo (Texto a memorizar/leer)	Apoyo Visual en Pantalla
0:00 - 0:25 (25 seg)	1. Problema (Presentador 1)	"Cuando un servidor en la nube recibe un pico masivo de tráfico, la asignación de recursos debe ser perfecta y en fracciones de segundo. El problema radica en que los algoritmos estáticos tradicionales fracasan en este punto; son rígidos, de diseño manual y colapsan cuando los datos operativos cambian de forma impredecible."	Visual: Animación rápida de un servidor o gráfico de tráfico rebasando el límite. Texto en grande: EL PROBLEMA: Algoritmos Estáticos vs Entornos Caóticos.
0:25 - 0:50 (25 seg)	2. Solución e Hipótesis (Presentador 2)	"La hipótesis de este proyecto plantea que el software puede evolucionar para escribir su propio código. El desarrollo propone un sistema de Programación Genética Tipada que, en lugar de evadir el desorden, se alimenta de él. Al combinar la transferencia de aprendizaje con una penalización estricta al crecimiento del código, el sistema generará de forma autónoma ecuaciones matemáticas limpias y adaptables."	Visual: Animación de un árbol sintáctico formándose. Texto en grande: LA SOLUCIÓN: Programación Genética Tipada + Control Geométrico.
0:50 - 1:05 (15 seg)	3. Objetivos (Presentador 3)	"El objetivo principal fue desarrollar un motor evolutivo capaz de resolver el Problema de la Mochila multi-instancia bajo estrés extremo. Todo esto garantizando reglas de decisión generalizables, minimizando el consumo de memoria y erradicando el código basura redundante."	Visual: <i>Bullet points</i> concisos con iconos. 1. Motor Evolutivo. 2. Generalización Multi-Instancia. 3. Cero Código Basura.
1:05 - 1:50 (45 seg)	4. Metodología (Presentador 1)	"Para el desarrollo, se utilizó el framework DEAP en Python simulando veinte fases de estrés continuo. La población genética fue sometida a mochilas y objetos totalmente variables. La clave metodológica fue la restricción: se limitó la altura del árbol sintáctico a 8 niveles y se implementó un castigo aritmético por longitud. Además, se descartaron intencionalmente técnicas de perfilamiento previo, exponiendo al algoritmo al entorno crudo para forzar una generalización real."	Visual: Capturas breves del código en Python (DEAP), enfocando la línea del <code>gp.staticLimit</code> . Un ícono tachando herramientas de preprocesamiento (K-Means).
1:50 - 2:35 (45 seg)	5. Resultados (Presentador 2)	"Los resultados demostraron una resiliencia algorítmica rotunda. A través del análisis de datos, las gráficas de dispersión evidenciaron que, ante un choque ambiental, el motor recupera la estabilidad en menos de tres generaciones. Además, el modelo de árbol predictivo comprobó matemáticamente que las heurísticas evolucionadas logran saturar la ganancia máxima de manera sistemática, mientras que la penalización contrajo con éxito el caos, creando expresiones matemáticamente óptimas."	Visual: Muestra la gráfica verde de "Convergencia Evolutiva". Luego, cambia rápidamente a una toma parcial del Árbol J48 de WEKA resaltando los nodos superiores.
2:35 - 3:00 (25 seg)	6. Conclusiones (Presentador 3)	"En conclusión, la evolución de software es superior a la creación estática. El sistema logró una inteligencia artificial verdaderamente autónoma que produce código transparente, rápido y sin consumo excesivo de memoria. Delegar la lógica operativa a un motor evolutivo se confirma como un paradigma viable para optimizar microservicios y gestionar las infraestructuras de alto valor del mañana. Gracias."	Visual: Toma de todos los presentadores. Texto final: CONCLUSIÓN: El futuro del software es evolutivo, no estático. Nombres de los integrantes del proyecto.

User prompt: Bien. Pero necesito una presentación más formal. De unos 10 segundos y seguirlo con lo que me estás proporcionando.

Response: Generando los ajustes para que el video tenga una apertura institucional y profesional...

Para cumplir con los **3 minutos exactos** de la rúbrica, he recalculado los tiempos. La introducción formal dura exactamente **10 segundos**, lo que deja **2 minutos y 50 segundos** para el resto del contenido técnico.

Aquí tienes el guion final definitivo:

Guion de Video: Tesis en 3 Minutos (3MT)

Tema: Optimización Dinámica y Evolución de Reglas de Asignación

Tiempo	Secciones	Diálogo (Texto a memorizar/leer)	Apoyo Visual en Pantalla
0:00 - 0:10 (10 seg)	INTRODUCCIÓN FORMAL	"Buen día. Se presenta el reporte final de la investigación titulada: ' Optimización Dinámica y Evolución de Reglas de Asignación mediante Programación Genética Tipada '."	Visual: Escudo de la institución, título de la tesis y nombre de los integrantes.
0:10 - 0:35 (25 seg)	1. Problema	"Cuando un servidor en la nube recibe un pico masivo de tráfico, la asignación de recursos debe ser perfecta y en fracciones de segundo. El problema radica en que los algoritmos estáticos tradicionales fracasan; son rígidos, de diseño manual y colapsan cuando los datos operativos cambian de forma impredecible."	Visual: Gráfico de tráfico rebasando el límite. Texto: EL PROBLEMA: Algoritmos Estáticos vs Entornos Caóticos.
0:35 - 1:00 (25 seg)	2. Solución e Hipótesis	"La hipótesis de este proyecto plantea que el software puede evolucionar para escribir su propio código. El desarrollo propone un sistema de Programación Genética Tipada que se alimenta del desorden. Al combinar transferencia de aprendizaje con una penalización estricta al crecimiento del código, el sistema genera de forma autónoma ecuaciones matemáticas limpias y adaptables."	Visual: Animación de un árbol sintáctico formándose. Texto: LA SOLUCIÓN: Programación Genética Tipada + Control Geométrico.
1:00 - 1:15 (15 seg)	3. Objetivos	"El objetivo principal fue desarrollar un motor evolutivo capaz de resolver el Problema de la Mochila multi-instancia bajo estrés extremo, garantizando reglas de decisión generalizables, minimizando el consumo de memoria y erradicando el código basura redundante."	Visual: Icons: 1. Motor Evolutivo. 2. Generalización. 3. Eficiencia de Memoria.
1:15 - 1:55 (40 seg)	4. Metodología	"Para el desarrollo, se utilizó el framework DEAP en Python simulando veinte fases de estrés continuo. La población genética fue sometida a mochilas y objetos totalmente variables. La clave metodológica fue la restricción: se limitó la altura del árbol a 8 niveles y se implementó un castigo aritmético por longitud. Se descartaron intencionalmente técnicas de perfilamiento previo para forzar una generalización real ante el entorno."	Visual: Capturas del código DEAP enfocando <code>gp.staticLimit</code> . Ícono tachando a "K-Means".
1:55 - 2:35 (40 seg)	5. Resultados	"Los resultados demostraron una resiliencia algorítmica rotunda. Las gráficas de dispersión evidenciaron que, ante un choque ambiental, el motor recupera la estabilidad en menos de tres generaciones. El modelo de árbol predictivo J48 comprobó matemáticamente que las heurísticas evolucionadas logran saturar la ganancia máxima, mientras que la penalización contrajo con éxito el caos, creando expresiones matemáticas óptimas."	Visual: Gráfica verde de "Convergencia Evolutiva" (Fase 5) y captura del Árbol de Decisión J48 de WEKA.
2:35 - 3:00 (25 seg)	6. Conclusiones	"En conclusión, la evolución de software es superior a la creación estática. El desarrollo logró una inteligencia artificial autónoma que produce código transparente y rápido. Delegar la lógica operativa a un motor evolutivo es el paradigma más viable para gestionar las infraestructuras de alto valor del mañana. Gracias."	Visual: Toma de todo el equipo. Texto: CONCLUSIÓN: El futuro del software es evolutivo.

Notas para la grabación:

- 1. La Introducción (0:00-0:10):** Debe decirse con voz pausada, muy clara y con postura formal.
- 2. Sincronización:** Si al grabar notan que la **Metodología** les toma más tiempo, pueden decir "se limitó la estructura del árbol" en lugar de "se limitó la altura del árbol sintáctico a 8 niveles" para ganar un par de segundos.
- 3. Imagen de Resultados:** Asegúrate de que cuando hables del "Árbol predictivo J48", la imagen en pantalla sea la de los nodos superiores que analizamos (donde se ve `Max_Ganancia`), ya que es la parte más impresionante visualmente.

¡Tu slide deck sobre **Optimización Dinámica y Evolución de Reglas de Asignación** está listo! Si necesitas algún cambio en los diálogos o las descripciones visuales, házmelo saber.

User prompt: Bien. Ahora proporcióname un título pensando que este código:

```
import operator
import random
import multiprocessing
import numpy as np
import pandas as pd
from deap import base, creator, tools, gp, algorithms
from knapsack import Item, KnapsackState, KnapsackInstance

# Identificador de la iteración final (Se actualiza en el bucle principal)
FASE_ACTUAL = 1

# Definición de funciones matemáticas seguras para el árbol
# CLASES DE MODELADO PARA LA SIMULACIÓN DEL PROBLEMA DE MOCHILA
class Item:
    def __init__(self, id_item, weight, profit):
        self.id = id_item
        self.weight = weight
        self.profit = profit
        self.ratio = profit / weight

class KnapsackState:
    def __init__(self, capacity):
        self.capacity = capacity
        self.current_weight = 0.0
        self.current_profit = 0.0

def pack(self, item):
    if self.current_weight + item.weight <= self.capacity:
        self.current_weight += item.weight
        self.current_profit += item.profit

class KnapsackInstance:
    def __init__(self, id_instancia, capacity, items):
        self.id = id_instancia
        self.capacity = capacity
        self.items = items

# PARÁMETROS Y CONFIGURACIÓN GLOBALES
FASE_ACTUAL = 1
NUM_GENERACIONES_POR_FASE = 20
# Número de ciclos evolutivos por fase de estrés
MAX_TREE_HEIGHT = 8

# Control estructural del 'Bloat'
# Definición de funciones matemáticas seguras para el árbol
def div_segura(izq, der):
    if der == 0:
        return 1.0
    # Previene la división por cero asignando un valor neutral
    if abs(der) < 1e-6:
        return 1.0
    return izq / der

# Configuración de la Programación Genética (GP)
pset = gp.PrimitiveSet("MAIN", 3)
# Define tres variables de entrada para la fórmula generada
# Configuración de la Programación Genética (GP) / 4 operadores básicos + ratio beneficio/peso
pset = gp.PrimitiveSet("MAIN", 3)
```

```

pset.addPrimitive(operator.add, 2) pset.addPrimitive(operator.sub, 2) pset.addPrimitive(operator.mul, 2) pset.addPrimitive(div_segura, 2)
pset.renameArguments(ARG0='P', ARG1='W', ARG2='PW') # Creacion de las estructuras para maximizar la aptitud (Fitness)
creator.create("FitnessMax", base.Fitness, weights=(1.0,)) # Indica que el objetivo es Maximizar el valor creator.create("Individual",
gp.PrimitiveTree, fitness=creator.FitnessMax) # Crea el molde del arbol genetico # Creación de las estructuras para maximizar la aptitud
(Fitness) creator.create("FitnessMax", base.Fitness, weights=(1.0,)) creator.create("Individual", gp.PrimitiveTree, fitness=creator.FitnessMax) #
Registro de herramientas fundamentales de DEAP toolbox = base.Toolbox() toolbox.register("expr", gp.genHalfAndHalf, pset=pset, min =1,
max =3) toolbox.register("individual", tools.initliterate, creator.Individual, toolbox.expr) toolbox.register("population", tools.initRepeat, list,
toolbox.individual) toolbox.register("population", tools.initRepeat, list, toolbox.individual) # Registro de población toolbox.register("compile",
gp.compile, pset=pset) # Funcion de evaluacion de la hiper-heuristica (Simulacion de empaquetado en una instancia) # Funcion de evaluacion
de la hiper-heuristica (Simulación de empaquetado en una instancia) def evaluar_hiper_heuristica(individuo, instancia): #Evalúa una regla
matemática generada por GP contra una instancia específica. #Retorna el Fitness ajustado (Ganancia - Penalización). rutina_puntuacion =
toolbox.compile(expr=individuo) # Compila el árbol en una funcion ejecutable try: rutina_puntuacion = toolbox.compile(expr=individuo) except
Exception: return (-np.inf,) mochila = KnapsackState(capacity=instancia.capacity) items_puntuados = [] for item in instancia.items: # La IA
califica el objeto usando su formula generada (P, W, PW) puntuacion = rutina_puntuacion(item.profit, item.weight, item.ratio)
items_puntuados.append((puntuacion, item)) items_puntuados.sort(key=lambda x: x[0], reverse=True) # Ordena por puntuación descendente
try: puntuacion = rutina_puntuacion(item.profit, item.weight, item.ratio) items_puntuados.append((puntuacion, item)) except Exception: continue
# Empaquetado físico respetando la capacidad de la mochila items_puntuados.sort(key=lambda x: x[0], reverse=True) for puntuacion, item in
items_puntuados: mochila.pack(item) # Penalización por longitud del árbol (Bloat penalty): Controla el consumo de memoria y complejidad
penalizacion_longitud = len(individuo) * 0.01 return mochila.current_profit - penalizacion_longitud, return mochila.current_profit -
penalizacion_longitud, # Retorna la ganancia ajustada # Operadores Genéticos y control de crecimiento toolbox.register("select",
tools.selTournament, tournsize=3) toolbox.register("mate", gp.cxOnePoint) toolbox.register("mutate", gp.mutUniform, expr=toolbox.expr,
pset=pset) # Operadores Geneticos y control de crecimiento toolbox.register("select", tools.selTournament, tournsize=3) # Selecciona padres
mediante torneo toolbox.register("mate", gp.cxOnePoint) # Realiza el cruce toolbox.register("mutate", gp.mutUniform, expr=toolbox.expr,
pset=pset) # Introduce variabilidad genética # Limita la altura máxima del árbol en cruce y mutación para controlar el Bloat estructural
toolbox.decorate("mate", gp.staticLimit(key=operator.attrgetter("height"), max_value=8)) toolbox.decorate("mutate",
gp.staticLimit(key=operator.attrgetter("height"), max_value=8)) # Decoração para limitar la altura máxima del árbol (Control de Bloat Estructural)
toolbox.decorate("mate", gp.staticLimit(key=operator.attrgetter("height"), max_value=MAX_TREE_HEIGHT)) toolbox.decorate("mutate",
gp.staticLimit(key=operator.attrgetter("height"), max_value=MAX_TREE_HEIGHT)) # Generador Estocastico de bases de datos (Simula el
entorno cambiante) def generar_base_datos_aleatoria(num_instancias=10, num_objetos=50): #Genera un lote masivo y aleatorio de
problemas de mochila para la prueba. instancias = [] for i in range(num_instancias): capacidad = random.uniform(50.0, 150.0) @@ -83,76
+104,80 @@ def generar_base_datos_aleatoria(num_instancias=10, num_objetos=50): instancias.append(KnapsackInstance(f"Inst_{i}",
capacidad, objetos)) return instancias # Función principal del motor evolutivo (Sin KMeans; enfocado en la robustez de las múltiples instancias)
def clasificar_y_evolucionar(lista_instancias, generaciones=20): #Ejecuta el proceso evolutivo sobre un conjunto completo de instancias para
obtener un fitness promedio robusto. print("Iniciando evolucion con enfoque multi-instancia sin diagnostico previo.") # Configuración de las
estadísticas: Captura Promedio, Max_Ganancia y Desviación Estándar (Smoothed Analysis) estadisticas = tools.Statistics(lambda ind:
ind.fitness.values[0]) # Mide el rendimiento promedio de la poblacion # Función principal del motor evolutivo con Transferencia de Aprendizaje
(Seeding) def clasificar_y_evolucionar(lista_instancias, generaciones=NUM_GENERACIONES_POR_FASE, poblacion_inicial_elite=None):
global FASE_ACTUAL if poblacion_inicial_elite: print("\nIniciando Fase con Transferencia de Aprendizaje (Seeding)") poblacion =
list(poblacion_inicial_elite) # Rellenar la población para mantener el tamaño deseado while len(poblacion) < 50:
poblacion.append(toolbox.individual()) else: print("\nIniciando Fase con Población Genética Aleatoria") poblacion = toolbox.population(n=50) #
Configuraciones de las estadísticas y el elitismo estadisticas = tools.Statistics(lambda ind: ind.fitness.values[0])
estadisticas.register("Promedio", np.mean) estadisticas.register("Max_Ganancia", np.max) # Rastrea la mejor regla encontrada hasta la fecha
estadisticas.register("Desviacion", np.std) # Mide la estabilidad y homogeneización de las reglas en la población # Herramienta para guardar al
mejor individuo historico (Elitismo) estadisticas.register("Max_Ganancia", np.max) estadisticas.register("Desviacion", np.std) salon_fama =
tools.HallOfFame(1) # PROCESO DE VALIDACIÓN ROBUSTA POR INSTANCIA # PROCESO DE EVALUACIÓN MASIVA POR INSTANCIA
(Robustez del Fitness) print("Evaluando el rendimiento de cada individuo contra todas las instancias.") resultados_por_individuo = {} #
Diccionario para guardar los resultados de fitness en todas las instancias. for instancia in lista_instancias: # Registra la funcion de evaluacion
por esta instancia especifica toolbox.register("evaluate", evaluar_hiper_heuristica, instancia=instancia) # Evalua cada individuo contra esta
instancia para recolectar resultados individuales results = [] for ind in toolbox.population(n=50): # Evaluamos toda la población contra esta única
instancia. fitness_tuple = evaluar_hiper_heuristica(ind, instancia) # Calculo manual y directo de fitness. results.append(fitness_tuple) # Nota: Si
se desea registrar el rendimiento en este punto, se haría aquí. # Evaluamos toda la población contra esta única instancia para obtener un
Fitness robusto promedio. for ind in poblacion: fitness_tuple = evaluar_hiper_heuristica(ind, instancia) results.append(fitness_tuple)
print("Evaluacion masiva de instancias completada.") print("Evaluación masiva de instancias completada.") # Ejecucion del ciclo generacional
completo usando eaSimple para la optimizacion final. # Ejecución del ciclo generacional completo (eaSimple) poblacion_final, bitacora =
algorithms.eaSimple( toolbox.population(n=50), toolbox, cxpb=0.7, mutpb=0.2, ngen=generaciones, stats=estadisticas, halloffame=salon_fama,
verbose=False) # Ejecucion del ciclo de generaciones sin logs en consola poblacion, toolbox, cxpb=0.7, mutpb=0.2, ngen=generaciones,
stats=estadisticas, halloffame=salon_fama, verbose=False) # Análisis Post-Ejecución y Logging de Resultados print("\n Resultados guardados
del analisis") mejor_individuo = salon_fama[0] # El individuo ganador es la mejor regla encontrada. mejores_reglas = {} # Se guarda la regla
ganadora generalizada de la fase actual. mejores_reglas[FASE_ACTUAL] = str(mejor_individuo) # Automatización de exportación a CSV
usando Pandas para la trazabilidad print("\nRESULTADOS DE LA FASE") mejor_individuo = salon_fama[0] df_log = pd.DataFrame(bitacora)
df_log.to_csv(f"Fase{FASE_ACTUAL}_bitacora_global.csv", index=False) print(f"Log de la convergencia guardado en:
Fase{FASE_ACTUAL}_bitacora_global.csv") filename = f"Fase{FASE_ACTUAL}_bitacora_global.csv" df_log.to_csv(filename, index=False)
print(f"[LOG] Trazabilidad de convergencia guardada en: {filename}") # Guardado automatico de las formulas ganadoras en archivo de texto
plano with open(f"Fase{FASE_ACTUAL}_mejores_reglas.txt", "w") as archivo_texto: archivo_texto.write(f"REPORT DE FORMULAS
EVOLUTIVAS - FASE {FASE_ACTUAL}\n") for cluster_id, regla in mejores_reglas.items(): archivo_texto.write(f"Mejor regla
encontrada:\n{regla}\n\n") archivo_texto.write(f"Mejor hiper-heurística generada:\n") archivo_texto.write(str(mejor_individuo) + "\n\n") return
mejores_reglas # Retorna las reglas ganadoras de la fase return salon_fama[0] # Bloque de ejecucion principal automatizado # BLOQUE DE
EJECUCIÓN PRINCIPAL AUTOMATIZADO Y ESCALABLE if __name__ == "__main__": for nueva_fase in range(1, 11): # Bucle configurado
para iniciar en Fase 1 y correr hasta 10. print(f" INICIANDO SIMULACIÓN DE FASE {nueva_fase}") FASE_ACTUAL = nueva_fase # Actualiza
la variable global de seguimiento poblacion_elite = None for nueva_fase in range(1, 21): # Ejecuta hasta Fase 20 para simular la Generación 2
y más. FASE_ACTUAL = nueva_fase # Generacion de datos estocásticos masivos para el experimento (10 instancias x 50 objetos)
base_de_datos = generar_base_datos_aleatoria(num_instancias=10, num_objetos=50) # Ejecucion del motor: Ahora usa la funcionalidad de
múltiples instancias por fase. clasificar_y_evolucionar(base_de_datos, generaciones=20) print("Todas las iteraciones simuladas han finalizado
exitosamente.") # Ejecución de la fase con transferencia de aprendizaje si hay un ganador anterior mejor_individuo_actual =
clasificar_y_evolucionar(base_de_datos, generaciones=NUM_GENERACIONES_POR_FASE, poblacion_inicial_elite=poblacion_elite) #
Actualizar la población elite para la siguiente fase (Elitismo) if mejor_individuo_actual is not None: # Conservamos solo el individuo más apto
como semilla para garantizar que no se pierda la optimización lograda. poblacion_elite = [mejor_individuo_actual] print("\nSIMULACIÓN DE
EVOLUCIÓN CONTINUA FINALIZADA.") Es antiguo y se actualizó por el siguiente. import operator import random import numpy as np import
pandas as pd from deap import base, creator, tools, gp, algorithms # DEAP: base/creator = estructuras, tools = operadores, # Parámetros
principales de la simulación NUM_FASES = 10 # entornos distintos a recorrer GEN_POR_FASE = 5 # generaciones por fase
TAM_POBLACION = 50 # individuos por generación TAM_ELITE = 10 # semilla transferida entre fases MAX_TREE_HEIGHT = 8 #
límite de altura, controla bloat NUM_INSTANCIAS = 10 # instancias knapsack por fase NUM_OBJETOS = 50 # objetos por instancia
FASE_ACTUAL = 1 # Clases del problema knapsack class Item: # Objeto candidato: peso, ganancia y ratio P/W (argumento PW del árbol)
def __init__(self, id_item, weight, profit): self.id = id_item self.weight = weight self.profit = profit self.ratio = profit / weight

```



```

class KnapsackState: # Estado de la mochila: acumula peso y ganancia al empacar def __init__(self, capacity): self.capacity =
capacity self.current_weight = 0.0 self.current_profit = 0.0 def pack(self, item): if self.current_weight + item.weight <=
self.capacity: self.current_weight += item.weight self.current_profit += item.profit class KnapsackInstance: # Problema
completo: una mochila con su lista de objetos def __init__(self, id_instancia, capacity, items): self.id = id_instancia self.capacity
= capacity self.items = items # Configuración del árbol GP: 4 operadores binarios + 3 argumentos (P, W, PW) def div_segura(izq, der):
return izq / der if abs(der) > 1e-6 else 1.0 pset = gp.PrimitiveSet("MAIN", 3) pset.addPrimitive(operator.add, 2) pset.addPrimitive(operator.sub,
2) pset.addPrimitive(operator.mul, 2) pset.addPrimitive(div_segura, 2) pset.renameArguments(ARG0='P', ARG1='W', ARG2='PW') # P=profit,
W=weight, PW=ratio # Estructuras DEAP: fitness de maximización e individuo árbol GP if not hasattr(creator, "FitnessMax"):
creator.create("FitnessMax", base.Fitness, weights=(1.0,)) if not hasattr(creator, "Individual"): creator.create("Individual", gp.PrimitiveTree,
fitness=creator.FitnessMax) # Toolbox: generación, compilación y operadores evolutivos toolbox = base.Toolbox() toolbox.register("expr",
gp.genHalfAndHalf, pset=pset, min_=1, max_=3) toolbox.register("individual", tools.initIterate, creator.Individual, toolbox.expr)
toolbox.register("population", tools.initRepeat, list, toolbox.individual) toolbox.register("compile", gp.compile, pset=pset) # Selección
torneo, cruce de subárboles, mutación uniforme toolbox.register("select", tools.selTournament, tournsize=3) toolbox.register("mate",
gp.cxOnePoint) toolbox.register("mutate", gp.mutUniform, expr=toolbox.expr, pset=pset) # Decoradores: descartan árboles que excedan
MAX_TREE_HEIGHT toolbox.decorate("mate", gp.staticLimit(key=operator.attrgetter("height"), max_value=MAX_TREE_HEIGHT))
toolbox.decorate("mutate", gp.staticLimit(key=operator.attrgetter("height"), max_value=MAX_TREE_HEIGHT)) def evaluar_robusto(individuo,
lista_instancias): # Fitness = ganancia promedio sobre todas las instancias, penalizado por tamaño del árbol try: rutina_puntuacion =
toolbox.compile(expr=individuo) except Exception: return (-np.inf,) ganancias = [] for instancia in lista_instancias: mochila =
KnapsackState(capacity=instancia.capacity) items_puntuados = [] for item in instancia.items: try: score =
rutina_puntuacion(item.profit, item.weight, item.ratio) items_puntuados.append((score, item)) except Exception:
continue items_puntuados.sort(key=lambda x: x[0], reverse=True) for _, item in items_puntuados: mochila.pack(item)
ganancias.append(mochila.current_profit) if not ganancias: return (-np.inf,) penalizacion = len(individuo) * 0.01 return
(np.mean(ganancias) - penalizacion,) def registrar_evaluador(lista_instancias): # Vincula evaluador a las instancias de la fase actual
toolbox.register("evaluate", evaluar_robusto, lista_instancias=lista_instancias) def
generar_base_datos_aleatoria(num_instancias=NUM_INSTANCIAS, num_objetos=NUM_OBJETOS): # Genera instancias aleatorias para
simular entorno cambiante instancias = [] for i in range(num_instancias): capacidad = random.uniform(50.0, 150.0) objetos =
[Item(j, random.uniform(1.0, 20.0), random.uniform(10.0, 100.0)) for j in range(num_objetos)]
instancias.append(KnapsackInstance(f"Inst_{i}", capacidad, objetos)) return instancias def clasificar_y_evolucionar(lista_instancias,
generaciones=GEN_POR_FASE, elite_anterior=None): global FASE_ACTUAL # Seeding: elite heredada + nuevos individuos para
diversidad if elite_anterior: poblacion = [toolbox.clone(ind) for ind in elite_anterior] while len(poblacion) < TAM_POBLACION:
poblacion.append(toolbox.individual()) else: poblacion = toolbox.population(n=TAM_POBLACION)
registrar_evaluador(lista_instancias) # Invalidar fitness heredado: nuevo entorno, nueva evaluación for ind in poblacion: del
ind.fitness.values estadísticas = tools.Statistics(lambda ind: ind.fitness.values[0] if ind.fitness.valid else -np.inf)
estadísticas.register("Promedio", np.mean) estadísticas.register("Max_Ganancia", np.max) estadísticas.register("Desviacion", np.std)
salon_fama = tools.HallOfFame(TAM_ELITE) # Iteraciones generacionales: evaluar → seleccionar → cruzar → mutar poblacion_final,
bitacora = algorithms.eaSimple( poblacion, toolbox, cxpb=0.7, mutpb=0.2, ngen=generaciones, stats=estadísticas,
halloffame=salon_fama, verbose=True ) # Exportar bitácora y mejor regla de la fase df_log = pd.DataFrame(bitacora)
df_log.insert(0, "fase", FASE_ACTUAL) df_log.to_csv(f"Fase{FASE_ACTUAL:02d}_bitacora.csv", index=False) mejor = salon_fama[0]
with open(f"Fase{FASE_ACTUAL:02d}_mejor_regla.txt", "w") as f: f.write(f"Fase {FASE_ACTUAL} - Mejor hiper-heurística\n")
f.write(str(mejor) + "\n\n") f.write(f"Fitness: {mejor.fitness.values[0]:.4f}\n") f.write(f"Nodos: {len(mejor)} | Altura: {mejor.height}\n")
return list(salon_fama) if __name__ == "__main__": import glob, os # Eliminar archivos de ejecuciones anteriores patrones =
["Fase*_bitacora.csv", "Fase*_mejor_regla.txt", "resumen_completo.csv"] for patron in patrones: for archivo in glob.glob(patron):
os.remove(archivo) pool_elite = None # Bucle principal: una fase = un entorno nuevo + ciclo evolutivo for nueva_fase in range(1,
NUM_FASES + 1): FASE_ACTUAL = nueva_fase base_de_datos = generar_base_datos_aleatoria() pool_elite =
clasificar_y_evolucionar( base_de_datos, generaciones=GEN_POR_FASE, elite_anterior=pool_elite ) #
Consolidar bitácoras de todas las fases frames = [] for f in range(1, NUM_FASES + 1): try:
frames.append(pd.read_csv(f"Fase{f:02d}_bitacora.csv")) except FileNotFoundError: pass if frames: resumen =
pd.concat(frames, ignore_index=True) resumen.to_csv("resumen_completo.csv", index=False) print(resumen.groupby("fase")
["Max_Ganancia"].max().to_string()) print("\nMejor individuo final:") print(pool_elite[0])

```

Response: Considerando la evolución, los componentes técnicos y la reestructuración metodológica que presenta la versión definitiva del código de Programación Genética, aquí tienes tres propuestas de títulos académicos formales, ordenadas según el enfoque que se le desee dar a la investigación:

Opción 1 (Enfoque General y de Ingeniería)

Optimización Dinámica y Evolución de Reglas de Asignación mediante Programación Genética Tipada con Mecanismos de Transferencia de Aprendizaje Sostenido

Opción 2 (Enfoque Teórico y Metodológico - Recomendado)

Evolución Autónoma de Hiper-Heurísticas Resilientes para el Problema de la Mochila Multi-Instancia bajo Entornos Cambiantes de Estrés Estocástico

Opción 3 (Enfoque en Control de Complejidad Computacional)

Modelado Evolutivo Generalizado: Regulación Estructural del Crecimiento de Código mediante Penalización de Parsimonia en Programación Genética

¿Por qué estos títulos representan fielmente el nuevo código?

- Transferencia de Aprendizaje Sostenido (Seeding):** El nuevo código implementa formalmente la transferencia de una población semillero (TAM_ELITE = 10) entre las diferentes fases dinámicas a través de la variable pool_elite, superando la inicialización en frío de la versión anterior.
- Mitigación Activa del Caos y Estabilidad (TAM_POBLACION = 50):** Al invalidar los valores de aptitud heredados en cada cambio de entorno (del ind.fitness.values), el título refleja la naturaleza adaptativa real del motor ante las perturbaciones.
- Control Estructural de Código (Bloat):** Resalta directamente el uso de los decoradores de restricción de altura (MAX_TREE_HEIGHT = 8) y la función evaluadora penalizada numéricamente por longitud.

User prompt: Pero que vaya con el tipo de "feax:" "updt" o algo así

Response: ¡Entendido perfectamente, Ángel! Lo que necesitas es utilizar la convención de **Conventional Commits** (o etiquetas de control de cambios) adaptada para la estructura del guion. Esto le da un orden visual excelente, simulando un historial de desarrollo de software o control de versiones.

Utilizaremos las siguientes etiquetas para marcar cada cambio de ritmo y sección en el video:

- feat: (Nueva característica o sección)
- docs: (Documentación, introducción o créditos)
- fix: (Solución al problema o planteamiento de hipótesis)
- test: (Metodología, pruebas y simulaciones realizadas)
- perf: (Resultados de rendimiento y métricas obtenidas)
- updt: (Conclusiones y actualizaciones finales del paradigma)

Aquí tienes el guion final definitivo con una apertura formal de 10 segundos, adaptado rigurosamente a la tercera persona e impersonal, y estructurado con etiquetas de desarrollo:

Guion de Video: Tesis en 3 Minutos (3MT)

Tema: Optimización Dinámica y Evolución de Reglas de Asignación

Tiempo	Commit / Sección	Diálogo (Texto a memorizar/leer)	Apoyo Visual en Pantalla
0:00 - 0:10 (10 seg)	docs: init_presentation	"Buen día. Se presenta el reporte final de la investigación titulada: 'Optimización Dinámica y Evolución de Reglas de Asignación mediante Programación Genética Tipada'. "	Visual: Escudo de la institución, título formal de la tesis y nombre de los integrantes del proyecto.
0:10 - 0:35 (25 seg)	feat: problem_statement	"Cuando un servidor en la nube recibe un pico masivo de tráfico, la asignación de recursos debe ser perfecta y en fracciones de segundo. El problema radica en que los algoritmos estáticos tradicionales fracasan; son rígidos, de diseño manual y colapsan cuando los datos operativos cambian de forma impredecible."	Visual: Gráfico de tráfico saturando un servidor y rebasando el límite. Texto: EL PROBLEMA: Algoritmos Estáticos vs Entornos Caóticos.
0:35 - 1:00 (25 seg)	fix: hypothesis_framework	"La hipótesis de este proyecto plantea que el software puede evolucionar para escribir su propio código. El desarrollo propone un sistema de Programación Genética Tipada que se alimenta del desorden. Al combinar transferencia de aprendizaje con una penalización estricta al crecimiento del código, el sistema genera de forma autónoma ecuaciones matemáticas limpias y adaptables."	Visual: Animación de un árbol sintáctico formándose a partir de nodos lógicos. Texto: LA SOLUCIÓN: Programación Genética Tipada + Control Geométrico.
1:00 - 1:15 (15 seg)	docs: target_objectives	"El objetivo principal fue desarrollar un motor evolutivo capaz de resolver el Problema de la Mochila multi-instancia bajo estrés extremo, garantizando reglas de decisión generalizables, minimizando el consumo de memoria y erradicando el código basura redundante."	Visual: Iconos concisos en pantalla: 1. Motor Evolutivo. 2. Generalización Completa. 3. Eficiencia de Memoria.
1:15 - 1:55 (40 seg)	test: methodology_env	"Para el desarrollo, se utilizó el framework DEAP en Python simulando veinte fases de estrés continuo. La población genética fue sometida a mochilas y objetos totalmente variables. La clave metodológica fue la restricción: se limitó la altura del árbol a 8 niveles y se implementó un castigo aritmético por longitud. Se descartaron intencionalmente técnicas de perfilamiento previo para forzar una generalización real ante el entorno."	Visual: Capturas del código DEAP enfocando la restricción <code>gp.staticLimit</code> . Un ícono bloqueando o tachando la palabra "K-Means".
1:55 - 2:35 (40 seg)	perf: evaluation_metrics	"Los resultados demostraron una resiliencia algorítmica rotunda. Las gráficas de dispersión evidenciaron que, ante un choque ambiental, el motor recupera la estabilidad en menos de tres generaciones. El modelo de árbol predictivo J48 comprobó matemáticamente que las heurísticas evolucionadas logran saturar la ganancia máxima, mientras que la penalización contrajo con éxito el caos, creando expresiones matemáticas óptimas."	Visual: Gráfica verde de "Convergencia Evolutiva" (Fase 5) y capturas del Árbol de Decisión J48 extraído de WEKA.
2:35 - 3:00 (25 seg)	updt: final_conclusions	"En conclusión, la evolución de software es superior a la creación estática. El desarrollo logró una inteligencia artificial autónoma que produce código transparente y rápido. Delegar la lógica operativa a un motor evolutivo es el paradigma más viable para gestionar las infraestructuras de alto valor del mañana. Gracias."	Visual: Toma de todo el equipo de trabajo. Texto de cierre: CONCLUSIÓN: El futuro del software es evolutivo.

💡 Recomendaciones para la edición de video con este formato:

- Puedes colocar las etiquetas (docs:, feat:, fix:) en una **esquina superior de la pantalla simulando una consola de comandos** o una terminal de Git. Cambiar la etiqueta en pantalla al mismo tiempo que cambia el presentador le dará un estilo visual sumamente innovador y muy ligado a las Tecnologías Computacionales.
- Mantén el ritmo controlado en la sección de test: y perf:, ya que contienen términos técnicos densos que el jurado debe asimilar con claridad.

User prompt: No. Ya no tengo que hacer el vídeo. El vídeo ya lo hice. Ahora necesito el título y descripción de la actualización del github

Response: Aquí se presentan las propuestas estructuradas bajo la convención formal de **Conventional Commits** (los estándares de la industria para el control de versiones en GitHub), adaptadas perfectamente al enfoque impersonal y de tercera persona que requiere el proyecto.

Opción 1: Enfoque en el cambio estructural principal (Remoción de K-Means)

Este título y descripción son ideales si el objetivo del commit es documentar el cambio arquitectónico más importante de la última versión.

- **Título del Commit:** refactor: remover agrupamiento K-Means para consolidar la generalización multi-instancia
- **Descripción del Commit:**
 - Se elimina por completo la etapa de preprocesamiento basada en clustering no supervisado.
 - El desarrollo ahora expone al algoritmo directamente al entorno dinámico estocástico sin sesgos previos.
 - Se delega la resiliencia y el entendimiento del contexto de forma exclusiva al motor evolutivo.
 - Los resultados de validación confirman la mitigación de sobreajuste local, logrando una hiper-heurística robus

Opción 2: Enfoque en la optimización del rendimiento y control de código (Bloat)

Esta opción resalta las mejoras de eficiencia en el uso de memoria y la limpieza en las fórmulas matemáticas obtenidas.

- **Título del Commit:** perf: optimizar el control de Bloat estructural y la eficiencia de memoria del servidor
- **Descripción del Commit:**
 - Se integra de forma estricta la penalización aritmética por longitud en la función de evaluación de aptitud.
 - Ajuste normativo mediante el decorador 'gp.staticLimit' para restringir la altura máxima del árbol sintáctico
 - Eliminación drástica de código basura redundante (intrones) en las expresiones matemáticas generadas.
 - Se reduce la complejidad topológica, garantizando tiempos de respuesta mínimos y total trazabilidad en el repc

Opción 3: Enfoque en la resiliencia y transferencia de aprendizaje (Seeding)

Esta alternativa es la mejor si se desea destacar cómo la población se adapta a los choques ambientales y hereda el conocimiento de fases previas.

- **Título del Commit:** feat: implementar transferencia de aprendizaje continuo mediante pool de población élite
- **Descripción del Commit:**
 - Se introduce el mecanismo de 'seeding' generacional mediante la preservación de los individuos más aptos entre
 - Modificación en el flujo para invalidar los valores de fitness heredados ante cada irrupción ambiental, forzar
 - Las pruebas demuestran una capacidad de auto-sanación heurística, logrando la estabilización del promedio en n
 - La base de la población general mantiene la diversidad necesaria para la exploración estocástica sin extinguir

Recomendación: Si se va a subir todo el bloque final de código consolidado, la **Opción 1** es la que mejor justifica la madurez metodológica del proyecto de cara a la evaluación de la tesis.

User prompt: Bien. Ahora tenemos este código ya 100% funcional con todos los requisitos excelentes:

```
import operator import random import numpy as np import pandas as pd from deap import base, creator, tools, gp, algorithms # DEAP: base/creator = estructuras, tools = operadores, # Parámetros principales de la simulación NUM_FASES = 10 # entornos distintos a recorrer GEN_POR_FASE = 5 # generaciones por fase TAM_POBLACION = 50 # individuos por generación TAM_ELITE = 10 # semilla transferida entre fases MAX_TREE_HEIGHT = 8 # límite de altura, controla bloat NUM_INSTANCIAS = 10 # instancias knapsack por fase NUM_OBJETOS = 50 # objetos por instancia FASE_ACTUAL = 1 # Clases del problema knapsack class Item: # Objeto candidato: peso, ganancia y ratio P/W (argumento PW del árbol) def __init__(self, id_item, weight, profit): self.id = id_item self.weight = weight self.profit = profit self.ratio = profit / weight class KnapsackState: # Estado de la mochila: acumula peso y ganancia al empacar def __init__(self, capacity): self.capacity = capacity self.current_weight = 0.0 self.current_profit = 0.0 def pack(self, item): if self.current_weight + item.weight <= self.capacity: self.current_weight += item.weight self.current_profit += item.profit class KnapsackInstance: # Problema completo: una mochila con su lista de objetos def __init__(self, id_instancia, capacity, items): self.id = id_instancia self.capacity = capacity self.items = items # Configuración del árbol GP: 4 operadores binarios + 3 argumentos (P, W, PW) def div_segura(izq, der): return izq / der if abs(der) > 1e-6 else 1.0 pset = gp.PrimitiveSet("MAIN", 3) pset.addPrimitive(operator.add, 2) pset.addPrimitive(operator.sub, 2) pset.addPrimitive(operator.mul, 2) pset.addPrimitive(div_segura, 2) pset.renameArguments(ARG0='P', ARG1='W', ARG2='PW') # P=profit, W=weight, PW=ratio # Estructuras DEAP: fitness de maximización e individuo árbol GP if not hasattr(creator, "FitnessMax"): creator.create("FitnessMax", base.Fitness, weights=(1.0,)) if not hasattr(creator, "Individual"): creator.create("Individual", gp.PrimitiveTree, fitness=creator.FitnessMax) # Toolbox: generación, compilación y operadores evolutivos toolbox = base.Toolbox() toolbox.register("expr", gp.genHalfAndHalf, pset=pset, min_=1, max_=3) toolbox.register("individual", tools.initIterate, creator.Individual, toolbox.expr) toolbox.register("population", tools.initRepeat, list, toolbox.individual) toolbox.register("compile", gp.compile, pset=pset) # Selección torneo, cruce de subárboles, mutación uniforme toolbox.register("select", tools.selTournament, tournsize=3) toolbox.register("mate",
```

```

gp.cxOnePoint) toolbox.register("mutate", gp.mutUniform, expr=toolbox.expr, pset=pset) # Decoradores: descartan árboles que excedan
MAX_TREE_HEIGHT toolbox.decorate("mate", gp.staticLimit(key=operator.attrgetter("height"), max_value=MAX_TREE_HEIGHT))
toolbox.decorate("mutate", gp.staticLimit(key=operator.attrgetter("height"), max_value=MAX_TREE_HEIGHT)) def evaluar_robusto(individuo,
lista_instancias): # Fitness = ganancia promedio sobre todas las instancias, penalizado por tamaño del árbol try: rutina_puntuacion =
toolbox.compile(expr=individuo) except Exception: return (-np.inf,) ganancias = [] for instancia in lista_instancias: mochila =
KnapsackState(capacity=instancia.capacity) items_puntuados = [] for item in instancia.items: try: score =
rutina_puntuacion(item.profit, item.weight, item.ratio) items_puntuados.append((score, item)) except Exception:
continue items_puntuados.sort(key=lambda x: x[0], reverse=True) for _, item in items_puntuados: mochila.pack(item)
ganancias.append(mochila.current_profit) if not ganancias: return (-np.inf,) penalizacion = len(individuo) * 0.01 return
(np.mean(ganancias) - penalizacion,) def registrar_evaluador(lista_instancias): # Vincula evaluador a las instancias de la fase actual
toolbox.register("evaluate", evaluar_robusto, lista_instancias=lista_instancias) def
generar_base_datos_aleatoria(num_instancias=NUM_INSTANCIAS, num_objetos=NUM_OBJETOS): # Genera instancias aleatorias para
simular entorno cambiante instancias = [] for i in range(num_instancias): capacidad = random.uniform(50.0, 150.0) objetos =
[Item(j, random.uniform(1.0, 20.0), random.uniform(10.0, 100.0)) for j in range(num_objetos)]
instancias.append(KnapsackInstance(f"Inst_{i}", capacidad, objetos)) return instancias def clasificar_y_evolucionar(lista_instancias,
generaciones=GEN_POR_FASE, elite_anterior=None): global FASE_ACTUAL # Seeding: elite heredada + nuevos individuos para
diversidad if elite_anterior: poblacion = [toolbox.clone(ind) for ind in elite_anterior] while len(poblacion) < TAM_POBLACION:
poblacion.append(toolbox.individual()) else: poblacion = toolbox.population(n=TAM_POBLACION)
registrar_evaluador(lista_instancias) # Invalidar fitness heredado: nuevo entorno, nueva evaluación for ind in poblacion: del
ind.fitness.values estadísticas = tools.Statistics(lambda ind: ind.fitness.values[0] if ind.fitness.valid else -np.inf)
estadísticas.register("Promedio", np.mean) estadísticas.register("Max_Ganancia", np.max) estadísticas.register("Desviacion", np.std)
salon_fama = tools.HallOfFame(TAM_ELITE) # Iteraciones generacionales: evaluar → seleccionar → cruzar → mutar poblacion_final,
bitacora = algorithms.eaSimple( poblacion, toolbox, cxbp=0.7, mutpb=0.2, ngen=generaciones, stats=estadísticas,
halloffame=salon_fama, verbose=True ) # Exportar bitácora y mejor regla de la fase df_log = pd.DataFrame(bitacora)
df_log.insert(0, "fase", FASE_ACTUAL) df_log.to_csv(f"Fase{FASE_ACTUAL:02d}_bitacora.csv", index=False) mejor = salon_fama[0]
with open(f"Fase{FASE_ACTUAL:02d}_mejor_regla.txt", "w") as f: f.write(f"Fase {FASE_ACTUAL} - Mejor hiper-heurística\n")
f.write(str(mejor) + "\n\n") f.write(f"Fitness: {mejor.fitness.values[0]:.4f}\n") f.write(f"Nodos: {len(mejor)} | Altura: {mejor.height}\n")
return list(salon_fama) if __name__ == "__main__": import glob, os # Eliminar archivos de ejecuciones anteriores patrones =
["Fase*_bitacora.csv", "Fase*_mejor_regla.txt", "resumen_completo.csv"] for patron in patrones: for archivo in glob.glob(patron):
os.remove(archivo) pool_elite = None # Bucle principal: una fase = un entorno nuevo + ciclo evolutivo for nueva_fase in range(1,
NUM_FASES + 1): FASE_ACTUAL = nueva_fase base_de_datos = generar_base_datos_aleatoria() pool_elite =
clasificar_y_evolucionar( base_de_datos, generaciones=GEN_POR_FASE, elite_anterior=pool_elite ) #
Consolidar bitácoras de todas las fases frames = [] for f in range(1, NUM_FASES + 1): try:
frames.append(pd.read_csv(f"Fase{f:02d}_bitacora.csv")) except FileNotFoundError: pass if frames: resumen =
pd.concat(frames, ignore_index=True) resumen.to_csv("resumen_completo.csv", index=False) print(resumen.groupby("fase")
["Max_Ganancia"].max().to_string()) print("\nMejor individuo final:") print(pool_elite[0]) Tenemos que modificarlo para hacer las siguientes
funciones: Con base a las heurísticas siguientes: Resolver el GP n veces Ajustar parámetros Explorar valores Operadores (parámetros) Hay
que hacer las siguientes modificaciones: Población Tam = [5, 50, 100] Mut [0, 0.1, 0.5, 0.7, 1] Gen [5, 20, 50, 100] Cruce [0, 0.1, 0.5, 0.7, 1]
Fases (réplicas) [5, 20, 50, 100] Score: guardar 5 scores y usar mediana Pc, score, P (obtener sus parámetros) Maximization Check
interacciones Correr réplicas de fase. ----- Graficar el comportamiento (esto al final después de correr el código con las
combinaciones pertinentes, es decir, en teoría son como 5 códigos diferentes) Proporcionamelos para hacer test y coloca un comentario en
cada modificación para hacerlo manualmente en cada lista. Ejemplo: Población Tam = [5, 50, 100] #Aquí se modifican la población Mut [0, 0.1,
0.5, 0.7, 1] #Aquí se modifica la mutación

```

Response: Para lograr este nivel de experimentación (búsqueda de hiperparámetros o *Grid Search*), la mejor práctica en ingeniería de software no es tener 5 archivos enormes casi idénticos, sino tener **un solo código maestro automatizado**.

He reestructurado tu código para convertirlo en un **Banco de Pruebas Automático (Testbench)**. En la parte superior encontrarás el bloque de configuración manual. Utilizando la librería *itertools*, el código generará las combinaciones automáticamente.

¿Cómo probar "5 códigos diferentes" manualmente? Simplemente deja **un solo valor** por defecto en las listas que no quieras evaluar, y coloca los múltiples valores en la lista que sí quieres probar. El código ejecutará las 5 réplicas independientes, obtendrá la mediana y guardará todo en un archivo CSV llamado `resultados_exploracion_gp.csv`. Al final del código, agregué la función automática para **graficar las interacciones**.

Aquí tienes el código maestro listo para ejecutar:

```

import operator
import random
import numpy as np
import pandas as pd
import itertools
import statistics
import os
import matplotlib.pyplot as plt
import seaborn as sns
from deap import base, creator, tools, gp, algorithms

# =====
# 1. BLOQUE DE EXPLORACIÓN DE HIPERPARÁMETROS (TESTING MANUAL)
# =====
# INSTRUCCIÓN: Para probar un solo parámetro a la vez (como si fueran 5 códigos
# distintos), deja un solo valor por defecto en las listas que NO vayas a evaluar.

POBLACION_TAM_LIST = [5, 50, 100] # Aquí se modifica la población
MUT_PB_LIST = [0.0, 0.1, 0.5, 0.7, 1.0] # Aquí se modifica la mutación
GEN_LIST = [5, 20, 50, 100] # Aquí se modifican las generaciones
CRUCE_PB_LIST = [0.0, 0.1, 0.5, 0.7, 1.0] # Aquí se modifica el cruce
FASES_LIST = [5, 20, 50, 100] # Aquí se modifican las fases (entornos)

REPLICAS_SCORE = 5 # Guardar 5 scores independientes y usar la mediana

# Parámetros fijos

```

```

MAX_TREE_HEIGHT = 8
NUM_INSTANCIAS = 10
NUM_OBJETOS = 50

# =====
# 2. CLASES DEL PROBLEMA (MOCHILA)
# =====
class Item:
    def __init__(self, id_item, weight, profit):
        self.id = id_item
        self.weight = weight
        self.profit = profit
        self.ratio = profit / weight

class KnapsackState:
    def __init__(self, capacity):
        self.capacity = capacity
        self.current_weight = 0.0
        self.current_profit = 0.0

    def pack(self, item):
        if self.current_weight + item.weight <= self.capacity:
            self.current_weight += item.weight
            self.current_profit += item.profit

class KnapsackInstance:
    def __init__(self, id_instancia, capacity, items):
        self.id = id_instancia
        self.capacity = capacity
        self.items = items

# =====
# 3. CONFIGURACIÓN DEL ÁRBOL GP Y DEAP
# =====
def div_segura(izq, der):
    return izq / der if abs(der) > 1e-6 else 1.0

pset = gp.PrimitiveSet("MAIN", 3)
pset.addPrimitive(operator.add, 2)
pset.addPrimitive(operator.sub, 2)
pset.addPrimitive(operator.mul, 2)
pset.addPrimitive(div_segura, 2)
pset.renameArguments(ARG0='P', ARG1='W', ARG2='PW')

# Maximization (Verificación solicitada)
if not hasattr(creator, "FitnessMax"):
    creator.create("FitnessMax", base.Fitness, weights=(1.0,))
if not hasattr(creator, "Individual"):
    creator.create("Individual", gp.PrimitiveTree, fitness=creator.FitnessMax)

toolbox = base.Toolbox()
toolbox.register("expr", gp.genHalfAndHalf, pset=pset, min_=1, max_=3)
toolbox.register("individual", tools.initIterate, creator.Individual, toolbox.expr)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)
toolbox.register("compile", gp.compile, pset=pset)

toolbox.register("select", tools.selTournament, tournsize=3)
toolbox.register("mate", gp.cxOnePoint)
toolbox.register("mutate", gp.mutUniform, expr=toolbox.expr, pset=pset)

toolbox.decorate("mate", gp.staticLimit(key=operator.attrgetter("height"), max_value=MAX_TREE_HEIGHT))
toolbox.decorate("mutate", gp.staticLimit(key=operator.attrgetter("height"), max_value=MAX_TREE_HEIGHT))

# =====
# 4. FUNCIONES DE EVALUACIÓN Y ENTORNO
# =====
def evaluar_robusto(individuo, lista_instancias):
    try:
        rutina_puntuacion = toolbox.compile(expr=individuo)
    except Exception:
        return (-np.inf,)

    ganancias = []
    for instancia in lista_instancias:
        mochila = KnapsackState(capacity=instancia.capacity)
        items_puntuados = []
        for item in instancia.items:
            try:
                score = rutina_puntuacion(item.profit, item.weight, item.ratio)

```



```

        items_puntuados.append((score, item))
    except Exception:
        continue
    items_puntuados.sort(key=lambda x: x[0], reverse=True)
    for _, item in items_puntuados:
        mochila.pack(item)
    ganancias.append(mochila.current_profit)

if not ganancias:
    return (-np.inf,)

penalizacion = len(individuo) * 0.01
return (np.mean(ganancias) - penalizacion,)

def generar_base_datos_aleatoria(num_instancias=NUM_INSTANCIAS, num_objetos=NUM_OBJETOS):
    instancias = []
    for i in range(num_instancias):
        capacidad = random.uniform(50.0, 150.0)
        objetos = [Item(j, random.uniform(1.0, 20.0), random.uniform(10.0, 100.0))
                    for j in range(num_objetos)]
        instancias.append(KnapsackInstance(f"Inst_{i}", capacidad, objetos))
    return instancias

# =====
# 5. MOTOR EVOLUTIVO ADAPTADO PARA RECIBIR PARÁMETROS DINÁMICOS
# =====
def clasificar_y_evolucionar(lista_instancias, tam_poblacion, generaciones, cxpb, mutpb, elite_anterior=None):
    if elite_anterior:
        poblacion = [toolbox.clone(ind) for ind in elite_anterior]
        while len(poblacion) < tam_poblacion:
            poblacion.append(toolbox.individual())
    else:
        poblacion = toolbox.population(n=tam_poblacion)

    toolbox.register("evaluate", evaluar_robusto, lista_instancias=lista_instancias)

    for ind in poblacion:
        del ind.fitness.values

    estadisticas = tools.Statistics(lambda ind: ind.fitness.values[0] if ind.fitness.valid else -np.inf)
    estadisticas.register("Max_Ganancia", np.max)

    # Conservar top 10% como elite
    tam_elite_local = max(1, int(tam_poblacion * 0.1))
    salon_fama = tools.HallOfFame(tam_elite_local)

    algorithms.eaSimple(
        poblacion, toolbox,
        cxpb=cxpb, mutpb=mutpb,
        ngen=generaciones,
        stats=estadisticas,
        halloffame=salon_fama,
        verbose=False # Apagado para no saturar la consola en grid search
    )

    mejor_score = salon_fama[0].fitness.values[0]
    return list(salon_fama), mejor_score

# =====
# 6. ORQUESTADOR DE GRID SEARCH Y GRÁFICADOR DE INTERACCIONES
# =====
def ejecutar_grid_search():
    combinaciones = list(itertools.product(
        POBLACION_TAM_LIST, MUT_PB_LIST, GEN_LIST, CRUCE_PB_LIST, FASES_LIST
    ))

    resultados_totales = []
    print(f"Iniciando Grid Search. Total de combinaciones a probar: {len(combinaciones)}")

    for idx, (P_tam, Mut, Gen, Cx, Fases) in enumerate(combinaciones):
        print(f"[{idx+1}/{len(combinaciones)}] Probando -> Pob: {P_tam}, Mut: {Mut}, Gen: {Gen}, Cx: {Cx}, Fases: {Fases}")

        scores_replicas = []

        # Correr réplicas
        for replica in range(REPLICAS_SCORE):
            pool_elite = None
            score_final_replica = 0

```

```

# Recorrer fases de la réplica
for f in range(Fases):
    base_de_datos = generar_base_datos_aleatoria()
    pool_elite, score_fase = clasificar_y_evolucionar(
        lista_instancias=base_de_datos,
        tam_poblacion=P_tam,
        generaciones=Gen,
        cxpb=Cx,
        mutpb=Mut,
        elite_anterior=pool_elite
    )
    score_final_replica = score_fase # Guardamos el score de la última fase de la réplica

    scores_replicas.append(score_final_replica)

mediana_score = statistics.median(scores_replicas)

# Guardar resultados
resultados_totales.append({
    "Poblacion": P_tam,
    "Mutacion_Pb": Mut,
    "Generaciones": Gen,
    "Cruce_Pb": Cx,
    "Num_Fases": Fases,
    "Score_Rep1": scores_replicas[0],
    "Score_Rep2": scores_replicas[1],
    "Score_Rep3": scores_replicas[2],
    "Score_Rep4": scores_replicas[3],
    "Score_Rep5": scores_replicas[4],
    "Mediana_Score": mediana_score
})
print(f"    -> Mediana obtenida: {mediana_score:.2f}")

# Guardar en CSV para checar interacciones
df_resultados = pd.DataFrame(resultados_totales)
df_resultados.to_csv("resultados_grid_search_gp.csv", index=False)
print("\nResultados guardados exitosamente en 'resultados_grid_search_gp.csv'")

return df_resultados

def graficar_comportamiento_e_interacciones(df):
    """
    Check interacciones: Genera gráficos para visualizar cómo afectan
    los parámetros al score (Mediana).
    """
    sns.set_theme(style="whitegrid")

    # 1. Gráfico de pares (Pairplot) para ver correlaciones generales
    columnas_interes = ["Poblacion", "Mutacion_Pb", "Generaciones", "Cruce_Pb", "Num_Fases", "Mediana_Score"]
    df_interes = df[columnas_interes]

    plt.figure(figsize=(12, 10))
    sns.pairplot(df_interes, y_vars=["Mediana_Score"], x_vars=columnas_interes[:-1], kind="reg", height=4)
    plt.suptitle("Check de Interacciones: Impacto de Parámetros en la Mediana del Score", y=1.05)
    plt.savefig("interacciones_parametros.png", bbox_inches='tight')
    plt.show()

    # 2. Matriz de Correlación
    plt.figure(figsize=(8, 6))
    sns.heatmap(df_interes.corr(), annot=True, cmap="coolwarm", fmt=".2f")
    plt.title("Matriz de Correlación de Parámetros")
    plt.savefig("matriz_correlacion.png", bbox_inches='tight')
    plt.show()

if __name__ == "__main__":
    # 1. Ejecutar la búsqueda de hiperparámetros
    df_final = ejecutar_grid_search()

    # 2. Graficar el comportamiento final
    if len(df_final) > 1:
        print("\nGenerando gráficos de interacciones...")
        graficar_comportamiento_e_interacciones(df_final)
    else:
        print("\nSolo hay 1 combinación probada. Modifica las listas arriba para generar gráficos comparativos.")

```

¿Qué hace este código exactamente por ti?

1. **Evalúa N veces automáticamente:** Ejecutará todo el proceso evolutivo 5 veces (REPLICAS_SCORE = 5) para la configuración dada.

2. **Calcula la Mediana:** Obtiene los 5 scores finales (uno por cada réplica de fase) y les aplica `statistics.median()`.
3. **Guarda todos los Parámetros (Pc, score, P):** Registra el Cruce (Pc), la Mutación (Pm), la Población (P), los scores de las 5 rondas y la Mediana en el archivo `resultados_grid_search_gp.csv`.
4. **Grafica Comportamientos:** Una vez que termina de recorrer todas las combinaciones que le pusiste en las listas, utiliza Seaborn para leer el CSV y dibujar gráficas de regresión y una matriz de correlación (heatmap) guardándolas como imágenes .png. Esto es lo que necesitas para tu documento en la sección de "Check interacciones".

User prompt: >>> %Run GP_Test.py Traceback (most recent call last): File "C:\Users\jrhe0\proyecto-mochila-gp\src\GP_Test.py", line 9, in <module> import seaborn as sns ModuleNotFoundError: No module named 'seaborn' Me salió ese error

Response: Ese error es muy común y muy fácil de solucionar. Significa que tu entorno de Python no tiene instalada la librería seaborn, la cual agregamos al código en la última actualización para poder generar las gráficas estadísticas y la matriz de correlación (el mapa de calor) de forma automática al final de la simulación.

Para solucionarlo, solo necesitas instalar el paquete. Sigue estos pasos:

1. Abre tu terminal o línea de comandos (puede ser el *Símbolo del sistema* de Windows, *PowerShell*, o la terminal integrada que viene dentro de tu editor de código como VS Code o Thonny).
2. Escribe el siguiente comando y presiona **Enter**:

```
pip install seaborn
```
3. Espera unos segundos a que descargue e instale los archivos necesarios (al instalar seaborn, automáticamente se asegurará de que matplotlib y pandas estén en sus versiones correctas).

Una vez que termine el proceso y te indique que la instalación fue exitosa, vuelve a ejecutar tu archivo `GP_Test.py` y el error habrá desaparecido por completo.

User prompt: Traceback (most recent call last): File "C:\Users\jrhe0\proyecto-mochila-gp\src\GP_Test.py", line 9, in <module> import seaborn as sns ModuleNotFoundError: No module named 'seaborn' Ya hice lo que me dijiste y me da error

Response: Este es un clásico problema de "entornos divididos" en Python. Lo que ocurrió es que el comando que ejecutaste instaló la librería en la versión global de Python de tu computadora, pero tu editor de código está utilizando un entorno virtual distinto o su propia versión interna para correr el archivo.

Por la sintaxis de >>> %Run que aparecía en tu consola en pasos anteriores, es altamente probable que estés utilizando **Thonny** como tu editor. Este programa maneja sus librerías de forma independiente.

Tienes dos formas muy sencillas de solucionarlo definitivamente para que detecte seaborn:

Opción 1: Instalarlo usando la interfaz de tu editor (Recomendada para Thonny)

1. Ve al menú superior del programa y selecciona **Herramientas** (Tools).
2. Haz clic en la opción **Gestionar paquetes...** (Manage packages...).
3. En la barra de búsqueda de la ventana que se abre, escribe `seaborn` y presiona buscar o *Enter*.
4. Selecciona el paquete en la lista de resultados y haz clic en el botón **Instalar** (Install).
5. Una vez finalizado, cierra esa ventana y vuelve a correr tu código.

Opción 2: El método infalible (Instalación por código)

Si estás usando otro editor (como VS Code) o no encuentras el menú de paquetes, vamos a forzar la instalación utilizando exactamente el mismo entorno de Python que ejecuta tu código.

Abre tu archivo `GP_Test.py` y, justo en la **línea 1** (antes de todos tus `import`), pega este pequeño bloque:

```
import sys
import subprocess
subprocess.check_call([sys.executable, '-m', 'pip', 'install', 'seaborn'])
```

Al darle *Ejecutar* (Run), este código le dará la orden directa al sistema de descargar e instalar seaborn en la carpeta exacta donde lo está buscando.

Verás algo de texto de descarga en tu consola inferior. En cuanto termine y tu programa avance normalmente sin mostrar el error, **borra esas tres líneas** de tu archivo para que no intente descargar la librería cada vez que corras tu experimento.

User prompt: Bien. Ya está ejecutandose, son 1200 iteraciones al parecer

Response: ¡Excelente noticia! Efectivamente, el código está funcionando a la perfección.

Esas 1200 iteraciones son el resultado exacto de multiplicar todas las opciones que configuramos en las listas de hiperparámetros: **3** (Tamaños de Población) × **5** (Probabilidades de Mutación) × **4** (Generaciones) × **5** (Probabilidades de Cruce) × **4** (Fases) = **1200 combinaciones únicas**.

En tu captura se observa que el motor está calculando la mediana de manera impecable (1201.74), lo que confirma que las 5 réplicas por configuración están evaluando las instancias de la mochila, descartando anomalías y devolviendo el valor central estadístico tal como lo estipula tu metodología.

Un pequeño consejo para este paso: Dado que es una búsqueda exhaustiva (*Grid Search*) muy masiva (1200 combinaciones, cada una corriendo 5 réplicas que a su vez iteran múltiples fases y generaciones), este proceso va a exigir bastante poder de procesamiento y **tomará un buen tiempo en finalizar**.

Deja tu computadora trabajando ininterrumpidamente. Es completamente normal si los ventiladores comienzan a sonar un poco más rápido. Una vez que el contador llegue al [1200/1200], el sistema tabulará toda la información en el archivo `resultados_grid_search_gp.csv` y generará automáticamente tus gráficas de interacciones y matrices de correlación para el reporte final. ¡Ese CSV será oro puro para tu documentación!
